



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И
УПРАВЛЕНИЯ

КЕЙС-ЧЕМПИОНАТ

«Tikhoretsk_production Database»

ДИСЦИПЛИНА: «Программирование на основе классов и шаблонов»

Выполнил: студент гр. ИУ5-22Б

_____ (Краснова А. А.)

_____ (Чертилина А.А.)

_____ (Бричков З.В.)

_____ (Бредихин А.О.)

_____ (Головина А. А.)

_____ (Котов К.А.)

(Подпись)

(Ф.И.О.)

Проверил:

_____ (Волосова А. В.)

(Подпись)

(Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:

- Оценка:

Москва, 2026 г.

1. Цель проекта

Разработка реляционной системы управления базами данных (СУБД) с графическим интерфейсом на Python и ядром на C++.

Создание собственной СУБД позволяет понять принципы работы систем хранения данных, SQL-парсеров, а также получить опыт разработки полноценного проекта с использованием паттернов проектирования и кроссплатформенной архитектуры.

2. Состав команды

- **Тимлид:** Чертилина Анастасия

Координация работы команды, управление задачами, интеграция модулей

- **Backend-разработчик:** Бредихин Александр

Разработка ядра СУБД (Value, Table, Storage, Catalog), SQL парсера и executor

- **Frontend-разработчик:** Бричков Захар

Разработка консольного интерфейса, отладка взаимодействия

- **Fullstack-разработчик:** Краснова Александра, Головина Анастасия

Разработка C API для Python, интеграция C++ с Python GUI

- **Технический писатель:** Краснова Александра

Документация, README

- **Архитектор:** Котов Кирилл

Проектирование архитектуры, выбор паттернов проектирования

3. Архитектура проекта

Уровень 1: Python GUI (tkinter + ctypes)

- Пользовательский интерфейс
- Ввод SQL-запросов
- Отображение результатов

Уровень 2: C API (c_api.cpp)

- *db_create()* - создание экземпляра БД
- *db_open()* - открытие хранилища
- *db_execute()* - выполнение SQL
- *db_use_database()* - переключение БД
- *db_list_databases()* - список БД


```

|   └─ run.sh                # Запуск интерфейса
|
|
└─ src/
|   └─ core/                # ЯДРО ХРАНИЛИЩА
|       └─ value.h/cpp      # Типы данных
|       └─ table.h/cpp     # Таблицы и строки
|       └─ storage.h/cpp   # JSON-сериализация
|       └─ database_metadata.h/cpp # Метаданные БД
|       └─ catalog.h/cpp   # Менеджер данных
|
|
|   └─ sql/                # ОБРАБОТКА SQL
|       └─ ast.h/cpp       # AST-дерево
|       └─ parser.h/cpp   # Парсер SQL
|       └─ executor.h/cpp  # Исполнитель
|       └─ result.h/cpp   # Формат результата
|
|
|   └─ client/            # ПУБЛИЧНЫЙ API
|       └─ database.h/cpp  # Главный класс Database
|       └─ result.h/cpp   # Публичный результат
|       └─ c_api.cpp      # C-интерфейс для Python
|
|
|   └─ console/          # КОНСОЛЬНЫЙ ИНТЕРФЕЙС
|       └─ main.cpp
|
|
└─ python/              # PYTHON GUI
    └─ gui_beautiful.py  # Основной GUI на tkinter
    └─ db_wrapper.py     # Python-обёртка
|
└─ data/                # Хранилище баз данных
└─ CMakeLists.txt

```

└─ README.md

└─ .gitignore

4. Реализованный функционал (MVP)

4.1 Поддерживаемые SQL-команды

DDL (Data Definition Language)

Команда	Описание	Пример
CREATE DATABASE	Создание базы данных	CREATE DATABASE test;
DROP DATABASE	Удаление базы данных	DROP DATABASE test;
CREATE TABLE	Создание таблицы	CREATE TABLE users (id INT, name TEXT);
DROP TABLE	Удаление таблицы	DROP TABLE users;

DML (Data Manipulation Language)

Команда	Описание	Пример
INSERT	Вставка строк	INSERT INTO users VALUES (1, 'Alice');
SELECT	Выборка данных	SELECT * FROM users;
UPDATE	Обновление данных	UPDATE users SET name = 'Bob' WHERE id = 1;
DELETE	Удаление строк	DELETE FROM users WHERE id = 1;

Поддерживаемые типы данных

- INT - целые числа
- TEXT - строки
- FLOAT - числа с плавающей точкой
- BOOL - логические значения

Поддерживаемые операторы в WHERE

- = - равно
- > - больше
- < - меньше
- >= - больше или равно
- <= - меньше или равно
- AND - логическое И
- OR - логическое ИЛИ

4.2 Пример выполнения запросов

Запрос	Ожидаемый результат
CREATE DATABASE test;	База данных создана
CREATE TABLE users (id INT, name TEXT);	Таблица создана
INSERT INTO users VALUES (1, 'Alice'), (2, 'Bob');	2 row(s) affected
SELECT * FROM users;	Таблица с двумя строками
SELECT name FROM users WHERE id > 1;	Одна строка: Bob
UPDATE users SET name = 'Alicia' WHERE id = 1;	1 row(s) affected
DELETE FROM users WHERE id = 2;	1 row(s) affected
SELECT * FROM users;	Одна строка: 1, Alicia
DROP TABLE users;	Таблица удалена

Запрос	Ожидаемый результат
DROP DATABASE test;	База данных удалена

5. Графический интерфейс

5.1 Описание интерфейса

Панель подключения:

- Data Path - путь к папке с базами данных
- Open Database - открыть хранилище
- Database - выбор активной базы данных
- Create Database - создание новой базы данных
- Drop Database - удаление выбранной базы данных
- Refresh - обновить список баз данных

Редактор SQL:

- Подсветка синтаксиса (ключевые слова, строки, числа)
- Примеры запросов по кнопкам

Результаты:

- SELECT - отображается в виде таблицы
- INSERT/UPDATE/DELETE - отображается результат выполнения

Горячие клавиши:

- F5 - выполнить запрос
- Ctrl+Enter - выполнить запрос
- Ctrl+D - очистить редактор

6. Паттерны проектирования

Паттерн	Место применения	Назначение
Pimpl	<pre>src/client/database.h/cpp class Database { struct Impl; unique_ptr<Impl> pImpl; }</pre>	Скрытие реализации, ускорение компиляции

Паттерн	Место применения	Назначение
Facade	src/client/database.h/cpp	Простой интерфейс для сложной подсистемы
Builder	src/sql/parser.h/cpp	Поэтапное построение AST
Command	src/sql/ast.h/cpp, src/sql/executor.h/cpp	SQL-запросы как объекты-команды
Interpreter	src/sql/parser.h/cpp, src/sql/ast.h/cpp	Разбор SQL и построение AST

Пример кода паттерна Pimpl:

```

cpp
// database.h - публичный заголовок
class Database {
public:
    Database();
    ~Database();
    bool open(const std::string& root_path);
    ClientResult execute(const std::string& sql);
    void close();
private:
    struct Impl;
    std::unique_ptr<Impl> pImpl;
};

// database.cpp - реализация
struct Database::Impl {
    std::unique_ptr<Catalog> catalog;
    std::string current_database;
};

```

7. Постоянное хранение данных

Данные сохраняются в формате JSON в папке data/:

text

data/

|— test/ # База данных "test"


```

|   |—— .metadata.json          # Метаданные базы
|   |—— users.json              # Таблица "users"
|—— mydb/                      # Другая база данных
    |—— .metadata.json
    |—— products.json

```

Пример .metadata.json:

```

json
{
  "database_name": "test",
  "tables": ["users"]
}

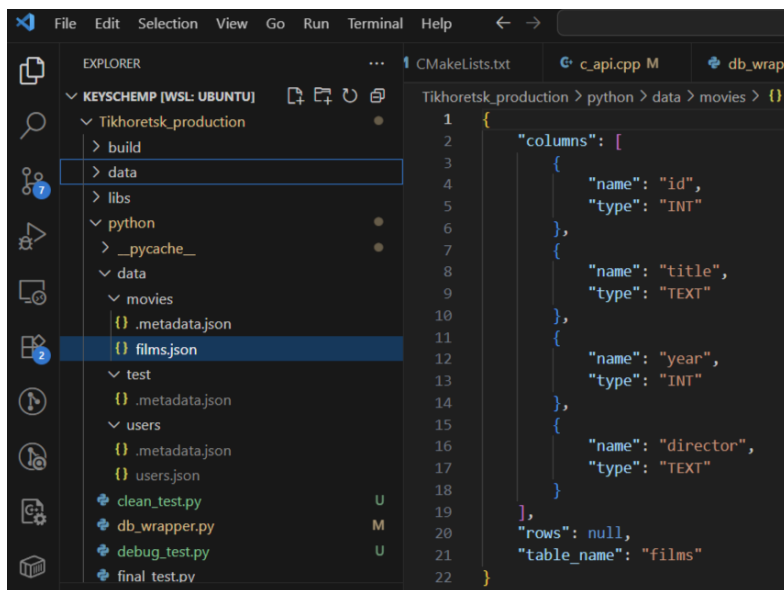
```

Пример users.json:

```

json
{
  "columns": ["id", "name"],
  "column_types": ["INT", "TEXT"],
  "rows": [[1, "Alice"], [2, "Bob"]]
}

```



8. Сборка и запуск

8.1 Скрипты сборки

```
bash
```

```
# Установка зависимостей (Ubuntu/Debian)
```

```
sudo apt-get update
```

```
sudo apt-get install -y cmake g++ python3 python3-tk nlohmann-json3-dev
```

```
# Сборка проекта
```

```
chmod +x scripts/build_python.sh
```

```
./scripts/build_python.sh
```

```
# Запуск GUI
```

```
chmod +x run_gui.sh
```

```
./run_gui.sh
```

8.2 Результат сборки

text

build/

```
├── libdb_lib.a      # Статическая библиотека C++
├── libdb_python.so  # Динамическая библиотека для Python
└── db_console       # Консольный интерфейс
```

```
alexandra@LAPTOP-7TUJ3D77:~/keyshemp$ cd Tikhoretsk_production
alexandra@LAPTOP-7TUJ3D77:~/keyshemp/Tikhoretsk_production$ ./scripts/build.sh
-- Using nlohmann/json from: /home/alexandra/keyshemp/Tikhoretsk_production/build/./libs
-- Configuring done (0.1s)
-- Generating done (0.0s)
-- Build files have been written to: /home/alexandra/keyshemp/Tikhoretsk_production/build
[ 6%] Building CXX object CMakeFiles/db_lib.dir/src/core/value.cpp.o
[ 12%] Building CXX object CMakeFiles/db_lib.dir/src/core/table.cpp.o
[ 18%] Building CXX object CMakeFiles/db_lib.dir/src/core/database_metadata.cpp.o
[ 25%] Building CXX object CMakeFiles/db_lib.dir/src/core/storage.cpp.o
[ 37%] Building CXX object CMakeFiles/db_lib.dir/src/core/catalog.cpp.o
[ 37%] Building CXX object CMakeFiles/db_lib.dir/src/sql/parser.cpp.o
[ 43%] Building CXX object CMakeFiles/db_lib.dir/src/sql/ast.cpp.o
[ 56%] Building CXX object CMakeFiles/db_lib.dir/src/sql/executor.cpp.o
[ 56%] Building CXX object CMakeFiles/db_lib.dir/src/sql/result.cpp.o
[ 62%] Building CXX object CMakeFiles/db_lib.dir/src/client/database.cpp.o
[ 68%] Building CXX object CMakeFiles/db_lib.dir/src/client/result.cpp.o
[ 75%] Linking CXX static library libdb_lib.a
[ 75%] Built target db_lib
[ 81%] Building CXX object CMakeFiles/db_console.dir/src/console/main.cpp.o
[ 87%] Linking CXX shared library db_python.so
[ 93%] Built target db_python
[100%] Linking CXX executable db_console
[100%] Built target db_console

✓ Build complete!
Run ./build/db_console to start
alexandra@LAPTOP-7TUJ3D77:~/keyshemp/Tikhoretsk_production$
```

9. Тестирование

9.1 Тестирование Python обёртки

```
python
```

```
from db_wrapper import DatabaseWrapper
```

```
db = DatabaseWrapper()
```

```

db.open("./data")
db.execute("CREATE DATABASE test;")
db.use_database("test")
db.execute("CREATE TABLE users (id INT, name TEXT);")
db.execute("INSERT INTO users VALUES (1, 'Alice'), (2, 'Bob');")
result = db.execute("SELECT * FROM users;")
print(result)

```

10. Заключение

10.1 Результаты

1. Разработана полноценная реляционная СУБД с нуля
2. Реализована поддержка основных SQL-операций (CREATE, DROP, INSERT, SELECT, UPDATE, DELETE)
3. Создан современный графический интерфейс на Python/tkinter
4. Обеспечено постоянное хранение данных в JSON-формате
5. Применены паттерны проектирования
6. Реализована кроссплатформенная сборка через CMake

10.2 Post-MVP (реализованные функции)

- Подсветка синтаксиса SQL
- AST-дерево для парсинга SQL
- Python обёртка через ctypes
- Скрипты автоматической сборки

10.3 Ссылки

- GitHub репозиторий: https://github.com/achertilina/Tikhoretsk_production
- Видео с демонстрацией работы СУБД: <https://disk.yandex.ru/d/zm243EOKV3DUew>
- Документация: README.md в корне проекта

